

Froxy – Arhitectură Proxy pentru Protocolul FTP

cu Facilități de Caching Atomic și Filtrare de Securitate

1 Introducere

În contextul actual al rețelelor de calculatoare, securitatea transferului de date și optimizarea lățimii de bandă reprezintă preocupări majore. Proiectul **Froxy** propune o soluție de tip *Application-Level Gateway* (Proxy) pentru protocolul FTP (File Transfer Protocol), având rolul de a intermedia conexiunile dintre clienții FTP standard și serverele externe.

Obiectivul principal al aplicației este dublu: asigurarea unui mecanism de control granular asupra traficului și îmbunătățirea timpilor de răspuns prin caching. Spre deosebire de o conexiune directă (End-to-End), Froxy introduce un punct centralizat de inspecție, permițând administratorilor să impună politici de securitate (restricții orare, filtrare de domenii, tipuri de fișiere permise).

De asemenea, prin implementarea unui sistem de stocare temporară (cache) a fișierelor frecvent accesate, aplicația reduce redundanța traficului de rețea și scade latența percepută de utilizatorul final. Proiectul abordează provocările specifice protocolului FTP, în special gestionarea conexiunilor duale (comenzi și date) și a modurilor de transfer (activ/pasiv).

2 Tehnologii Aplicate

Pentru dezvoltarea sistemului Froxy, am selectat un set de tehnologii care oferă control direct asupra resurselor de sistem și performanță ridicată:

- **Limbaajul C:** Alegerea a fost motivată de eficiența sa sporită și de faptul că API-ul POSIX pentru socket-uri (`sys/socket.h`) este nativ acestui limbaj, fiind standardul *de facto* pentru programarea de sistem în mediul UNIX/Linux. Utilizarea C permite un control granular asupra memoriei și a pointerilor, esențial pentru manipularea bufferelor de rețea la nivel scăzut fără overhead-ul claselor sau al abstracțiilor din limbajele de nivel înalt.
- **Protocolul TCP/IP:** FTP se bazează pe livrarea garantată a pachetelor și ordonarea corectă a fluxului de date. Utilizarea socket-urilor de tip `SOCK_STREAM` asigură fiabilitatea necesară transmiterii comenzilor critice și a integrității fișierelor transferate. O atenție deosebită a fost acordată gestionării fragmentării pachetelor TCP (*banner parsing*) pentru a preveni blocajele la conectare cauzate de mesajele lungi ale serverelor.
- **Concurență prin Multithreading (pthread):** Deoarece un proxy trebuie să servească multiple cereri simultan, am optat pentru un model de concurență bazat pe fire de execuție (*Thread-per-Client*). Această abordare permite izolarea contextului fiecărui utilizator și simplifică logica de blocare/așteptare a operațiilor I/O față de o abordare asincronă, fiind adecvată pentru scara proiectului.
- **Sistemul de Fișiere (File I/O):** Pentru implementarea mecanismului de caching, se utilizează funcții standard de I/O. Pentru a garanta consistența datelor în medii concurente, sistemul folosește scrierea în fișiere temporare și redenumirea atomică (`rename`) a acestora la finalizare.

3 Structura Aplicației

Arhitectura Froxy este una de tip *Man-in-the-Middle* (MitM), aplicația funcționând simultan ca server pentru clientul FTP și ca client pentru serverul FTP extern. Sistemul este compus din trei module logice interconectate:

1. **Modulul de Interceptare și Control:** Acționează ca punctul de intrare. Acceptă conexiuni pe portul standard (sau configurat), inițiază un fir de execuție nou pentru fiecare client și parsează comenzile primite.
Securitate: Aici se aplică logica de filtrare și sanitizare a input-ului. Înainte de procesarea oricărei comenzi critice (ex: `RETR`), se verifică existența secvențelor malițioase (Path Traversal: `../` sau `/`), iar cererile suspecte sunt respinse imediat.
2. **Modulul de Proxy și Date:** Gestionează complexitatea protocolului FTP. Acesta menține două canale de comunicație per sesiune:
 - *Canalul de Comandă:* Transmite transparent comenzile (`USER`, `PASS`, `CWD`) între client și server.
 - *Canalul de Date:* Interceptează negocierea porturilor (comenzile `PORT`/`PASV`). În modul pasiv, Froxy deschide un port propriu către client și se conectează la portul serverului real, transferând datele prin buffer-ul propriu.
3. **Modulul de Caching Atomic:** La detectarea unei comenzi de descărcare (`RETR`), acest modul verifică existența fișierului local.
Consistență: Pentru a rezolva problemele de concurență (*Race Conditions*), descărcarea nu se face direct în

fișierul destinație. Datele sunt scrise într-un fișier temporar (.tmp). Doar la finalizarea cu succes a transferului, se apelează funcția `rename()` pentru a face fișierul vizibil în cache.

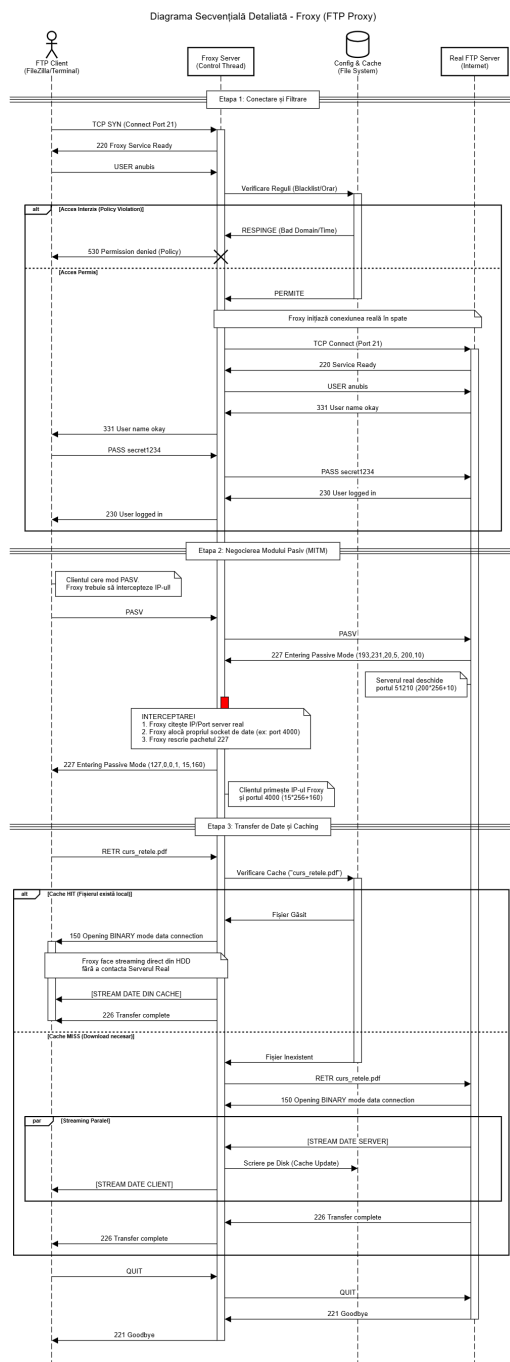


Figura 1. Diagrama Secvențială Detaliată - Froxy (FTP Proxy)

Fluxul detaliat al interacțiunii dintre module, inclusiv logica de interceptare a modului pasiv, este ilustrat în Figura 1.

4 Protocolul de Comunicare

Froxy implementează un subset al specificațiilor RFC 959, operând la nivelul Aplicație al stivei TCP/IP. Protocolul este bazat pe text, utilizând setul de caractere NVT-ASCII, cu comenzi terminate prin secvența <CRLF>.

Funcționalități implementate:

- **Conectare Robustă:** Implementarea unui buffer de acumulare dinamic și timeout pe socket pentru a citi corect bannerele de întâmpinare lungi sau fragmentate (ex: de la `ftp.gnu.org`), prevenind desincronizarea.
- **Autentificare:** Interceptarea comenzilor `USER` și `PASS`. Proxy-ul nu stochează credențiale, ci le transmite serverului destinație.
- **Managementul Sesiunii:** `QUIT` (închidere controlată), `TYPE` (setare mod binar/ascii).
- **Transfer de Date:**
 - **PASV:** Froxy modifică răspunsul serverului (227 Entering Passive Mode) înlocuind IP-ul și portul serverului cu propriile sale coordonate.
 - **RETR <fișier>:** Declanșează logica de caching atomic.
 - **STOR <fișier>:** Permite upload-ul (fără caching).
- **Răspunsuri:** Serverul Froxy generează răspunsuri standard FTP (ex: 220 Service Ready, 530 Permission Denied) pentru a asigura compatibilitatea cu orice client FTP (FileZilla, CLI, WinSCP).

5 Scenarii de Utilizare

Scenarii de Succes (Execuție Validă)

- **Acces Permis și Cache Miss (Cu scriere atomică):** Utilizatorul se conectează și cere fișierul `curs.pdf`. Froxy descarcă fișierul în `curs.pdf.tmp` și simultan îl trimite clientului. La final, fișierul temporar este redenumit atomic pentru a deveni valid în cache.
- **Acces Permis și Cache Hit:** Un al doilea utilizator cere `curs.pdf`. Froxy detectează fișierul valid în cache și îl livrează instantaneu, simulând protocolul FTP (150 Opening BINARY mode... 226 Transfer complete) fără a deschide o conexiune către serverul extern.
- **Configurare Admin:** Administratorul trimite comanda `RELOAD_CONFIG`. Froxy actualizează lista de domenii interzise fără a opri serviciul.

Scenarii de Eșec (Tratarea Erorilor)

- **Tentativă de Path Traversal:** Utilizatorul trimite comanda `RETR ../../../../etc/passwd`. Modulul de control detectează secvența interzisă și blochează cererea, returnând *550 Malicious filename detected*.
- **Restricție de Domeniu:** Utilizatorul încearcă conectarea la un server aflat în lista neagră. Froxy răspunde imediat cu codul *550 Access denied by policy* și închide conexiunea.
- **Server Indisponibil:** Clientul cere conectarea, dar serverul FTP țintă nu răspunde. Froxy capturează eroarea și trimite clientului *421 Service not available*.

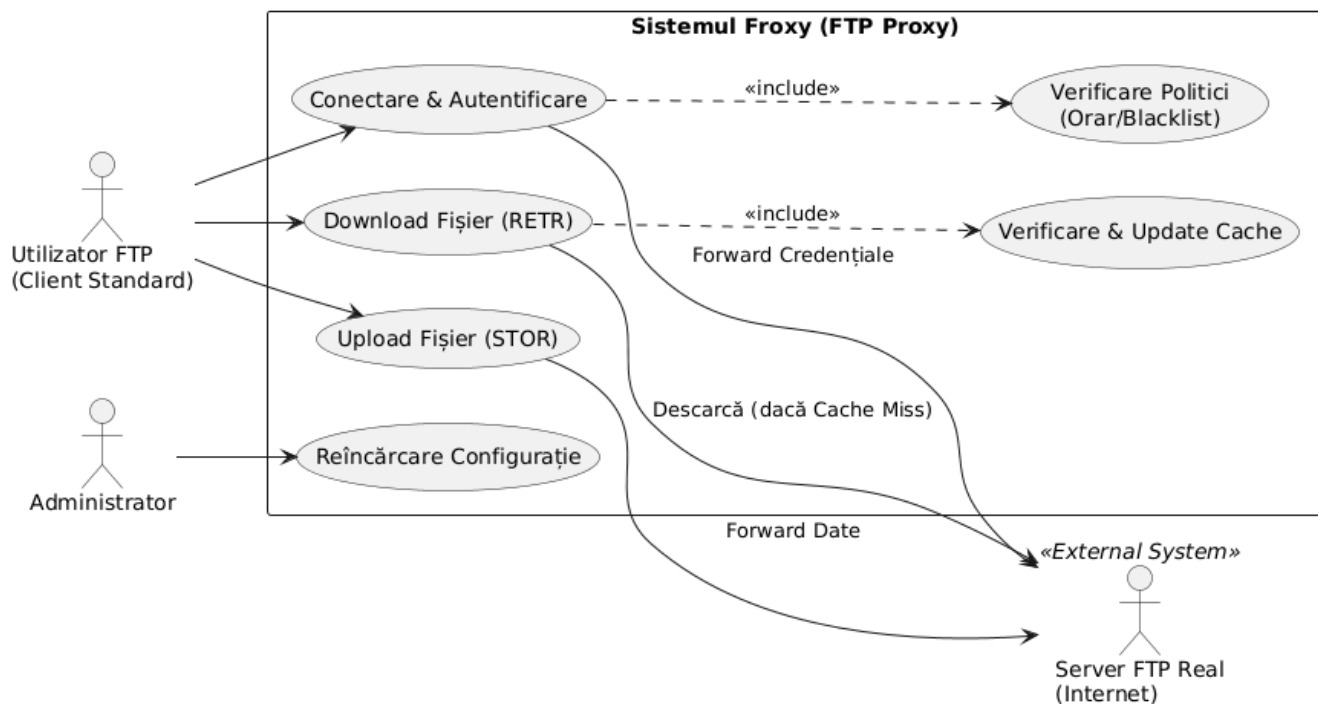


Figura 2. Diagram Use Case

6 Concluzii

Soluția propusă, Froxy, demonstrează viabilitatea implementării unui intermediar inteligent în comunicațiile FTP, oferind avantaje clare în medii controlate. Versiunea finală a proiectului a adresat probleme critice de programare concurentă, implementând **caching-ul atomic** pentru a garanta integritatea datelor și mecanisme de **sanitizare a input-ului** pentru securitate.

Ca direcții viitoare, soluția poate fi îmbunătățită prin adăugarea suportului pentru FTPS (FTP over SSL/TLS) și implementarea unui algoritm de înlocuire a cache-ului mai sofisticat (ex: LRU).

Bibliografie

1. Postel, J., & Reynolds, J. (1985). *File Transfer Protocol (FTP)*. RFC 959. Internet Engineering Task Force.
2. Stevens, W. R., Fenner, B., & Rudoff, A. M. (2004). *UNIX Network Programming, Volume 1: The Sockets Networking API* (3rd ed.). Addison-Wesley Professional.
3. Kerrisk, M. (2010). *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press.